

Trabajo Practico N°2

Verilog

Autor: Simón Saillen

Profesores: Ariel Pola, Agustin Pesce, Pablo Cayuela, Paul Romero,
Santiago Garaventta Pascual

Fundación Fulgor

Junio 2026

1. Introducción

Este informe tiene como objetivo documentar el desarrollo de un proyecto de diseño digital utilizando Verilog. El proyecto se centra en la implementación de un módulo TOP que integra los módulos VIO e ILA para control y depuración. Se busca validar el comportamiento del diseño mediante un testbench y realizar la implementación en una FPGA, conectándose remotamente a la placa.

2. Objetivos

- Incluir los módulos VIO e ILA para control y debug del módulo TOP.
- Realizar un testbench que valide el comportamiento del diseño sin los IP cores VIO e ILA.
- Implementar en FPGA conectándose remotamente a la placa.
- Realizar capturas de pantalla que muestren las waveforms del Testbench e ILA.

3. Desarrollo

3.1. Material proporcionado

Se proporcionaron cinco archivos base:

- `shiftdleds.v`: El modulo TOP que implementa una salida de leds ciclica.
- `shiftdleds_muxVioIla.v`: El mismo modulo TOP con los modulos VIO e ILA integrados.
- `tb_shiftdleds_file.v`: Testbench para el modulo TOP sin los modulos VIO e ILA.
- `Arty_Master_v2.xdc`: Archivo de constrains para los pines de la FPGA.
- `make_tb_files.py`: Script para generar los vectores de entrada para el testbench.

Los mismos fueron renombrados, cambiados y adaptados para mejor legibilidad y para cumplir con los objetivos del trabajo práctico.

3.2. Modificaciones realizadas

3.2.1. Módulo TOP.v

Se unificaron los archivos `shiftdleds.v` y `shiftdleds_muxVioIla.v` a `TOP.v` para tener un solo módulo que pueda ser simulado y sintetizado.

Se agregaron las instancias de los modulos VIO e ILA al final del archivo, y se agregaron los puertos de entrada y salida necesarios para controlar los modulos VIO e ILA desde el testbench.

Algunos nombres de señales fueron cambiados para mejorar la legibilidad del código.

3.2.2. Testbench `tb_TOP.v`

Se renombró el archivo `tb_shiftleds_file.v` a `tb_TOP.v`.

Sin modificar el modulo TOP, se cambió el parámetro del testbench `NB_COUNT` de 32 a 16, al hacer este cambio la cantidad necesaria de ciclos para superar el primer limite `R0` pasa de 2^{22} a 2^6 , permitiendo shifteos de registro mas seguidos y logrando una mejor observación del comportamiento.

Se modifco el path de los archivos para ser leídos desde la ubicación del testbench. Algunos nombres de señales fueron cambiados para mejorar la legibilidad del código.

3.2.3. Generador de Vectores `make_tb_files.py`

El script `make_file.py` fue renombrado a `make_tb_files.py` y modificado para generar los archivos de entrada en la ubicación del script.

Se corrigio un error en la generacion del archivo `reset.out` que hacia que el reset este siempre activo, impidiendo que el contador del modulo TOP pueda avanzar, ahora solo se activa un ciclo de reloj cada vez que se cambia el valor de `switch[3:0]` en el testbench, a excepcion del penultimo cambio de valor, donde se inhabilita el `init` para mantener el valor de salida de los leds previo al cambio, se omite el reset para observar la conservacion del ultimo valor del contador.

4. Resultados

4.1. Simulacion

Antes de modificar la cantidad de bits del contador, se realizó la simulación del testbench `tb_TOP.v` y se obtuvieron las siguientes waveforms:

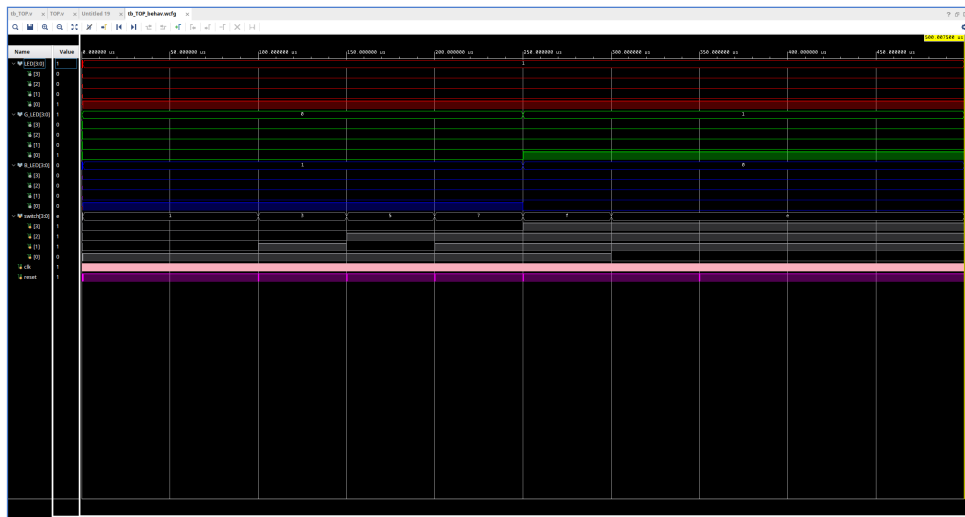


Figura 1: Simulación del testbench `tb_TOP.v` con contador de 32 bits.

Como se puede observar, el contador no logra superar el primer limite, manteniendo el valor de

salida de los leds en 4'b0001 de manera constante. Pero se puede comprobar que el color del led si cambia al cambiar el valor de `switch[3]` a 1.

Una vez modificado el parámetro del contador a 16 bits, se realizó nuevamente la simulación del testbench `tb_TOP.v` y se obtuvieron las siguientes waveforms:

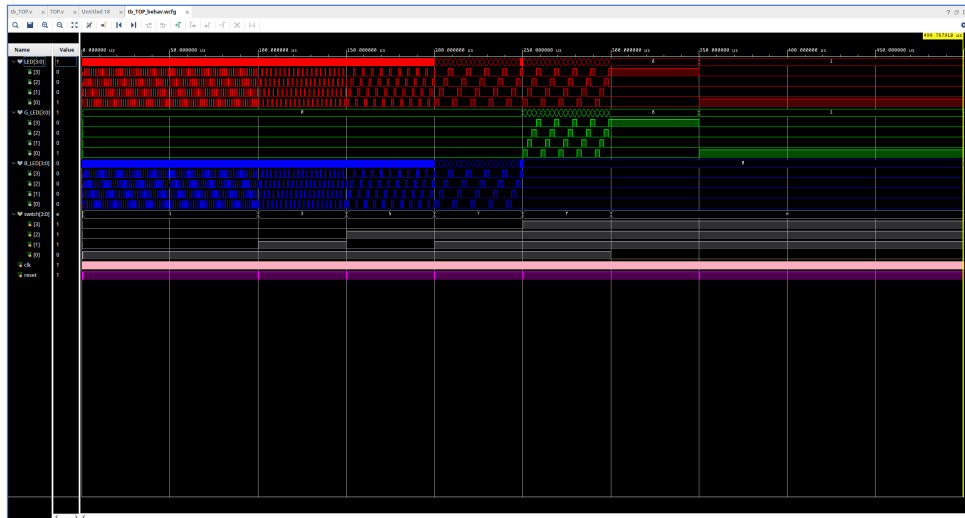


Figura 2: Simulación del testbench `tb_TOP.v` con contador de 16 bits.

Como se puede observar, el contador logra superar los distintos limites, cambiando el valor de salida de los leds de manera cíclica. Además, se puede observar el cambio de velocidad del ciclo de los leds al cambiar el valor de `switch[2:0]`. Y por último, se puede observar que al cambiar el valor de `switch[0]` a 0, se inhabilita el `init`, impidiendo que el contador pueda avanzar y manteniendo el valor de salida previo de los leds.

4.2. Implementación en FPGA

5. Conclusiones